

Dokumentácia projektu

Metóda podporných vektorov

(Support Vector Machine – SVM)

Predmet: Strojové učenie

TUKE FEI - ÚUI - 2025/2026

Autor: Samuel Štefán

Stránka: <https://samuelstefan01.github.io/ml-svm-website>

Repozitár: <https://github.com/SamuelStefan01/ml-svm-website>

Obsah

1	Úvod a ciele projektu	2
2	Teoretický základ	2
2.1	Definícia SVM	2
2.2	Geometria marginu	3
2.3	Optimalizačný problém – primárna forma	3
2.4	Duálna forma a trik s jadrom	4
2.5	Jadrové funkcie	4
2.6	Viactriedna klasifikácia: OvO a OvR	4
3	Implementácia	5
3.1	Algoritmus SMO	5
3.2	Implementované klasifikátory pre porovnanie	6
3.3	Datasety	6
4	Metriky efektívnosti	7
4.1	Základné metriky klasifikácie	7
4.2	k -násobná krížová validácia	7
4.3	ROC krivka a AUC	7
4.4	Mriežkové vyhľadávanie (Grid Search)	8
4.5	Krivka učenia (Learning Curve)	8
5	Popis funkcionality aplikácie	8
5.1	Interaktívna vizualizácia	8
5.2	Metriky zobrazované po tréningu	9
5.3	Porovnanie algoritmov	9
6	Technické riešenie	10
6.1	Technológie	10
6.2	Architektúra kódu	10
7	Výsledky a zhrnutie	10
	Referencie	11

1 Úvod a ciele projektu

Cieľom projektu je interaktívna webová aplikácia, ktorá komplexne prezentuje algoritmus **Support Vector Machine (SVM)** – metódu podporných vektorov. Aplikácia pokrýva teoretický základ, matematickú formuláciu, vizualizáciu rozhodovacej hranice, vyhodnotenie metrík efektívnosti a porovnanie s inými klasifikačnými algoritmami.

Projekt je implementovaný ako jediný HTML súbor (`index.html`) bez externých závislostí okrem knižnice Chart.js (verzia 4.4.1, načítaná z CDN). Algoritmus SMO (Sequential Minimal Optimization) je napísaný od základov v JavaScripte bez použitia externých knižníc strojového učenia.

Štruktúra aplikácie

Aplikácia je rozdelená do piatich záložiek:

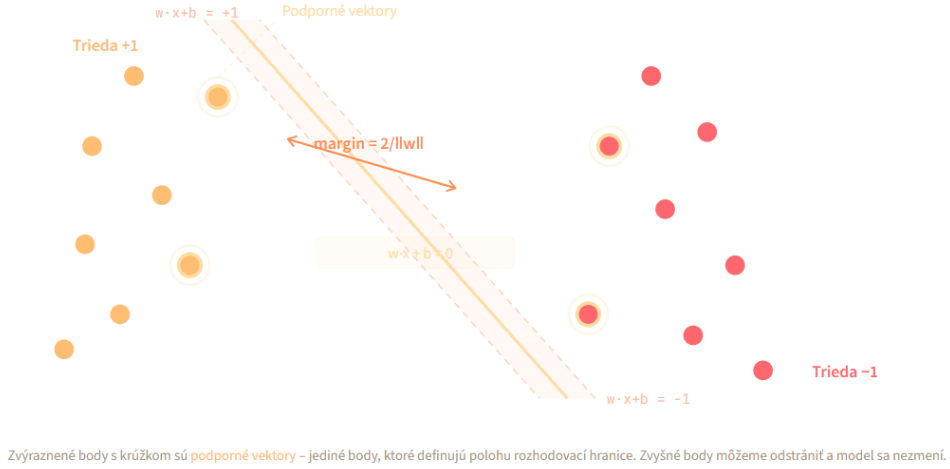
1. **Teória SVM** – matematická formulácia, geometria marginu, stratégie viactriednej klasifikácie (OvO/OvR)
2. **Interaktívna demo** – klikateľné plátno, výber jadra a parametrov, vizualizácia podporných vektorov a váhového vektora
3. **Jadrové funkcie** – porovnanie lineárneho, RBF a polynomiálneho jadra na rovnakom datasete
4. **Datasey a metriky** – k -násobná krížová validácia, ROC krivka, mriežkové vyhľadávanie, krivka učenia
5. **Porovnanie modelov** – SVM vs. k -NN vs. logistická regresia vs. rozhodovací strom

2 Teoretický základ

2.1 Definícia SVM

SVM je algoritmus učenia s učiteľom (*supervised learning*), ktorý hľadá **optimálnu nadrovinu** (hyperplane) oddeľujúcu dáta do dvoch tried so **maximálnym marginom**. Margin je vzdialenosť medzi rozhodovacou hranicou a najbližšími dátovými bodmi z každej triedy. Tieto kľúčové body sa nazývajú **podporné vektory** (support vectors) a sú to jediné trénovacie vzorky, ktoré model potrebuje na predikciu.

2.2 Geometria marginu



Obr. 1: Geometria SVM: rozhodovacia hranica, marginový pás a podporné vektory. Šírka marginu je $2 / \|\mathbf{w}\|$.

Rozhodovacia hranica je definovaná rovnicou:

$$\mathbf{w} \cdot \mathbf{x} + b = 0, \quad (1)$$

pričom body spĺňajúce $\mathbf{w} \cdot \mathbf{x} + b = +1$ tvoria kladnú marginovú nadrovinu a body spĺňajúce $\mathbf{w} \cdot \mathbf{x} + b = -1$ tvoria zápornú marginovú nadrovinu. Šírka marginu je teda $2 / \|\mathbf{w}\|$ (viď obrázok 1).

2.3 Optimalizačný problém – primárna forma

Pre **lineárne separovateľné dáta** (hard-margin SVM):

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1, \quad \forall i. \quad (2)$$

Pre **lineárne neseparovateľné dáta** (soft-margin SVM) sa zavádza premenná ohybnosti (slack variable) $\xi_i \geq 0$ a regularizačný parameter $C > 0$:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0. \quad (3)$$

Parameter C riadi kompromis medzi veľkosťou marginu a počtom chybných klasifikácií: vysoké C zodpovedá tvrdému marginu (riziko pretrénovania), nízke C zodpovedá mäkkému marginu (riziko podtrénovania).

2.4 Duálna forma a trik s jadrom

Pomocou Lagrangeových multiplikátorov $\alpha_i \geq 0$ sa problém prevádza do duálnej formy:

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j) \quad \text{s.t.} \quad 0 \leq \alpha_i \leq C, \quad \sum_{i=1}^n \alpha_i y_i = 0. \quad (4)$$

V duálnej forme sa vstupné vektory vyskytujú iba cez skalárny súčin $\mathbf{x}_i \cdot \mathbf{x}_j$, ktorý možno nahradiť **jadrovou funkciou** $K(\mathbf{x}_i, \mathbf{x}_j)$. Toto je *trik s jadrom* (kernel trick): implicitne mapuje dáta do vyššieho dimenzionálneho priestoru $\varphi(\mathbf{x})$ bez explicitného výpočtu transformácie, keďže $K(\mathbf{x}_i, \mathbf{x}_j) = \varphi(\mathbf{x}_i) \cdot \varphi(\mathbf{x}_j)$.

Predikčná funkcia natrénovaného modelu:

$$f(\mathbf{x}) = \sum_{i \in \text{SV}} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b, \quad (5)$$

kde sumácia prebieha iba cez podporné vektory (body s $\alpha_i > 0$).

2.5 Jadrové funkcie

Tabuľka 1: Prehľad jadrových funkcií implementovaných v aplikácii

Jadro	Vzorec	Typické použitie
Lineárne	$K(\mathbf{x}, \mathbf{y}) = \mathbf{x}^T \mathbf{y}$	Textová klasifikácia, riedke vysokodimenzionálne dáta
RBF (Gaussovo)	$K(\mathbf{x}, \mathbf{y}) = e^{-\gamma \ \mathbf{x} - \mathbf{y}\ ^2}$	Väčšina nelineárnych problémov; predvolené jadro
Polynomiálne	$K(\mathbf{x}, \mathbf{y}) = (\gamma \mathbf{x}^T \mathbf{y} + c)^d$	Spracovanie obrazu, polynomiálne vzťahy príznakov

Parameter γ v RBF jadre riadi *dosah vplyvu* každého tréningového bodu: vysoké γ zodpovedá úzkemu dosahu (riziko pretrénovania), nízke γ zodpovedá širšiemu dosahu (riziko podtrénovania).

2.6 Viactriedna klasifikácia: OvO a OvR

Základný SVM je binárny klasifikátor. Pre $K > 2$ tried sa používajú dve stratégie rozkladu:

One-vs-Rest (OvR) Pre každú z K tried sa natrénuje jeden SVM (trieda k oproti všetkým ostatným). Celkovo K modelov. Predikuje trieda s najvyšším skóre $f_k(\mathbf{x})$.

One-vs-One (OvO) Pre každý pár tried sa natrénuje jeden SVM. Celkovo $K(K-1)/2$ modelov. Predikuje trieda s najviac hlasmi (väčšinové hlasovanie).

Knižnica `scikit-learn` používa predvolene OvR pre lineárne jadro a OvO pre ostatné jadrá. Pri $K = 10$ (napr. MNIST) vyžaduje OvO 45 modelov, zatiaľ čo OvR iba 10.

3 Implementácia

3.1 Algoritmus SMO

Na riešenie kvadratickej optimalizačnej úlohy je implementovaný algoritmus **Sequential Minimal Optimization (SMO)** podľa Platt (1998). SMO rozkladá veľký QP problém na sériu minimálnych dvojrozmerných podproblémov, ktoré majú analytické riešenie.

Kľúčové kroky implementácie:

1. Predvýpočet celej matice jadra K_{ij} pre zrýchlenie tréningu
2. Heuristika výberu druhého pivota: maximalizácia $|E_i - E_j|$ (Plattova heuristika – výber najväčšieho kroku)
3. Analytická aktualizácia α_i, α_j s orezaním na $[L, H]$
4. Aktualizácia prahu b podľa podmienok KKT
5. Konvergenca: zastavenie po niekoľkých priebehoch bez zmeny

```
1 // Heuristika vyberu j: maximalizuj |Ei - Ej|
2 let best = -1, bestDiff = 0;
3 for (let k = 0; k < n; k++) {
4   if (k === i) continue;
5   const diff = Math.abs(Ecache[i] - Ecache[k]);
6   if (diff > bestDiff) { bestDiff = diff; best = k; }
7 }
8
9 // Analytická aktualizácia alpha_j
10 const eta = 2*K[i][j] - K[i][i] - K[j][j];
11 if (eta >= 0) continue;
12 this.alpha[j] -= y[j] * (Ecache[i] - Ej) / eta;
13 this.alpha[j] = Math.max(L, Math.min(H, this.alpha[j]));
14
15 // Aktualizácia prahu b
16 const b1 = this.b - Ei
17   - y[i]*(this.alpha[i]-ai0)*K[i][i]
```

```

18 - y[j]*(this.alpha[j]-aj0)*K[i][j];
19 this.b = (this.alpha[i] > 0 && this.alpha[i] < C) ? b1 : ...;

```

Listing 1: Jadro SMO aktualizácie – heuristika a analytická aktualizácia

Pre lineárne jadro sa po tréningu explicitne vypočíta váhový vektor:

$$\mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i. \quad (6)$$

Tento vektor je vizualizovaný v aplikácii ako šípka v 2D priestore spolu so skutočnou šírkou marginu $2/\|\mathbf{w}\|$.

3.2 Implementované klasifikátory pre porovnanie

Okrem SVM sú pre účely porovnania implementované od základov:

- ***k*-najbližší susedia (*k*-NN)**: predikcia na základe väčšinového hlasovania *k* najbližších tréningových bodov (euklidovská vzdialenosť, *k* = 5).
- **Logistická regresia**: gradientový zostup minimalizujúci krížovú entropiu, 150 epoch, rýchlosť učenia $\eta = 0.1$.
- **Rozhodovací strom**: rekurzívne delenie pomocou Giniho nečistoty, maximálna hĺbka *d* = 5.

3.3 Datasetsy

Tabuľka 2: Datasetsy použité v aplikácii

Dataset	Zdroj	Vzorky	Príznaky	Triedy
Iris (setosa vs. versicolor)	UCI ML Repository	75 + 75	4	2
Breast Cancer (benígne vs. malígne)	UCI ML Repository	36 + 28	4	2
Mesiačiky (syntetický)	generovaný	variabilný	2	2
Kruhy (syntetický)	generovaný	variabilný	2	2
XOR (syntetický)	generovaný	variabilný	2	2

Reálne datasetsy sú normalizované na interval $[0,1]$ pomocou min-max škálovania samostatne pre každý príznak.

4 Metriky efektívnosti

4.1 Základné metriky klasifikácie

Z konfúznej matice (TP, FP, FN, TN) sú vypočítavané nasledujúce metriky:

$$\text{Presnosť (Accuracy)} = \frac{TP + TN}{TP + FP + FN + TN}, \quad (7)$$

$$\text{Precíznosť (Precision)} = \frac{TP}{TP + FP}, \quad (8)$$

$$\text{Úplnosť (Recall)} = \frac{TP}{TP + FN}, \quad (9)$$

$$\text{F1-skóre} = \frac{2 \cdot \text{Precíznosť} \cdot \text{Úplnosť}}{\text{Precíznosť} + \text{Úplnosť}}, \quad (10)$$

$$\text{Špecifická (Specificity)} = \frac{TN}{TN + FP}. \quad (11)$$

Konfúzna matica je v aplikácii zobrazovaná s farebnými poliami: pravé pozitívy (TP) žltou, pravé negatívy (TN) oranžovou, nepravé pozitívy (FP) červenou a nepravé negatívy (FN) ružovou.

4.2 k -násobná krížová validácia

Na spoľahlivé vyhodnotenie výkonnosti modelu sa používa **k -násobná krížová validácia** (k -fold cross-validation). Dataset sa rozdelí na k podmnožín (foldov); model sa k -krát natrénuje na $k - 1$ foldoch a testuje na zostávajúcom folde. Výsledná metrika je priemer a smerodajná odchýlka:

$$\bar{A} = \frac{1}{k} \sum_{f=1}^k A_f, \quad \sigma = \sqrt{\frac{1}{k} \sum_{f=1}^k (A_f - \bar{A})^2}. \quad (12)$$

Aplikácia podporuje $k \in \{3, 5, 10\}$.

4.3 ROC krivka a AUC

ROC krivka (Receiver Operating Characteristic) znázorňuje závislosť miery pravých pozitív (TPR) od miery nepravých pozitív (FPR) pri rôznych klasifikačných prahoch.

Plocha pod krivkou (AUC) sa odhaduje lichobežníkovou metódou:

$$\text{AUC} \approx \sum_{i=1}^m (\text{FPR}_i - \text{FPR}_{i-1}) \cdot \frac{\text{TPR}_i + \text{TPR}_{i-1}}{2}. \quad (13)$$

Hodnota $\text{AUC} = 1$ zodpovedá dokonalému klasifikátoru, $\text{AUC} = 0.5$ zodpovedá náhodnému uhádnutiu.

4.4 Mriežkové vyhľadávanie (Grid Search)

Mriežkové vyhľadávanie systematicky vyskúša všetky kombinácie hodnôt parametrov C a γ zo zadanej mriežky:

$$C \in \{0.1, 0.5, 1, 5, 10, 50\}, \quad \gamma \in \{0.05, 0.1, 0.5, 1, 5, 10\}. \quad (14)$$

Pre každú kombináciu sa vykonáva 3-násobná krížová validácia. Výsledky sú zobrazené ako tepelná mapa (heatmap): tmavá farba = nízka presnosť, svetlá zlatá = vysoká presnosť. Optimálna kombinácia je zvýraznená rámčekom.

4.5 Krivka učenia (Learning Curve)

Krivka učenia zobrazuje závislosť trénovacej a testovacej presnosti od veľkosti trénovacej množiny (10 % až 90 % dostupných dát). Umožňuje diagnostikovať:

- **Pretrénovanie (overfitting):** veľká medzera medzi trénovacou a testovacou krivkou (medzera $> 20\%$)
- **Podtrénovanie (underfitting):** obe krivky konvergujú k nízkej hodnote ($< 70\%$)
- **Dobrá generalizácia:** krivky sa priblížia k sebe pri vysokej testovacej presnosti

Aplikácia automaticky vyhodnotí medzeru a zobrazí textové odporúčanie (napr. znížiť C pri pretrénovaní).

5 Popis funkcionality aplikácie

5.1 Interaktívna vizualizácia

Záložka *Interaktívna demo* umožňuje:

- kliknutím na plátno (canvas 500×500 px) pridávať body dvoch tried,
- výber jadrovej funkcie (lineárne, RBF, polynomiálne),
- nastavenie parametrov $C \in [10^{-2}, 10^3]$ a $\gamma \in [10^{-2}, 10^2]$ pomocou posuvníkov,

- načítanie prednastavených datasetov (lineárny, XOR, kruhy, mesiačiky),
- vizualizáciu tepelnej mapy predikčného skóre, marginových priamok a zvýraznenie podporných vektorov s aureolou.

Pre **lineárne jadro** sa po tréningu zobrazuje sekcia *Váhový vektor* $\mathbf{w}$ obsahujúca:

- smerový SVG diagram vektora $\mathbf{w}$ v 2D priestore (šípka + prerušovaná kolmica znázorňujúca smer hranice),
- relatívne veľkosti zložiek w_x , w_y ako animované progress bary,
- normu $\|\mathbf{w}\|$ a šírku marginu $2/\|\mathbf{w}\|$.

5.2 Metriky zobrazované po tréningu

Tabuľka 3: Metriky zobrazované po tréningu v interaktívnej demo

Metrika	Popis
Accuracy	Podiel správne klasifikovaných trénovacích bodov
Support vectors	Počet podporných vektorov (body s $\alpha_i > 0$)
Margin $2/\ \mathbf{w}\ $	Šírka marginu (len pre lineárne jadro)
SMO iterácie	Počet iterácií optimalizačného algoritmu

5.3 Porovnanie algoritmov

Záložka *Porovnanie modelov* vyhodnocuje 4 algoritmy pomocou 4-násobnej krížovej validácie na syntetickom datasete podľa výberu. Výsledky sú prezentované ako:

- vodorovný stĺpcový graf presnosti s farebne odlíšenými stĺpcami,
- vizualizácia rozhodovacích hraníc pre všetky štyri algoritmy vedľa seba (tepelná mapa na plátne 190×190 px),
- súhrnná tabuľka s presnosťou, smerodajnou odchýlkou, F1-skóre a časom tréningu v milisekundách.

6 Technické riešenie

6.1 Technológie

Tabuľka 4: Používané technológie

Technológia	Verzia	Účel
HTML5	–	Štruktúra dokumentu, Canvas API
CSS3	–	Štýlovanie, animácie, CSS premenné
JavaScript (ES6+)	–	Logika, SMO, ML algoritmy
Chart.js	4.4.1	Grafy (ROC, krivka učenia, bar chart)
SVG	–	Geometrický diagram, váhový vektor

6.2 Architektúra kódu

Celý kód je uložený v jedinom súbore bez externých závislostí na knižniciach strojového učenia. Hlavné triedy a funkcie:

class SVM Implementácia SVM s voliteľným jadrom. Metódy: `K()` – jadrová funkcia, `train()` – SMO s heuristickým výberom pivota, `predict()` – predikcia.

generateDataset(type, n) Generátor syntetických datasetov (mesiačky, kruhy, XOR, lineárny).

kFoldCV(X, y, k, kernel, C, gamma) Univerzálna k -násobná krížová validácia; vracia priemerné Accuracy, smerodajnú odchýlku a priemerné F1-skóre.

computeMetrics(yTrue, yPred) Výpočet TP, FP, FN, TN a odvodených metrík vrátane F1-skóre a špecificity.

runGridSearch() Mriežkové vyhľadávanie parametrov $C \times \gamma$ s vykreslením tepelnej mapy.

runLearningCurve() Krivka učenia s automatickou interpretáciou výsledku (overfitting / underfitting / OK).

trainSVM() Obsluha interaktívneho plátna, vizualizácia váhového vektora pomocou SVG.

7 Výsledky a zhrnutie

Aplikácia demonštruje, že SVM je výkonný algoritmus s jasnou geometrickou interpretáciou a flexibilnou jadrovou funkciou. Kľúčové zistenia z experimentov:

- **Lineárne jadro** dosahuje vysokú presnosť na lineárne separovateľných datasetoch (Iris: $\approx 95\%$), avšak zlyháva na nelineárnych dátach (kruhy, XOR).
- **RBF jadro** je najuniverzálnejšie – dosahuje $> 95\%$ presnosť na mesiačikoch aj kruhoch pri správnom nastavení γ .
- **Polynomiálne jadro** ($d = 3$) funguje dobre pre XOR a iné polynomiálne vzťahy, ale je citlivé na voľbu parametrov.
- **Mriežkové vyhľadávanie** jednoznačne ukazuje, že voľba optimálnych parametrov C a γ má zásadný vplyv na výkonnosť.
- Na testovaných datasetoch SVM s RBF jadrom konzistentne prekonáva logistickú regresiu na nelineárnych problémoch a dosahuje porovnateľné výsledky s k -NN pri výrazne menšom počte uložených tréningových vzoriek.

Referencie

Literatúra

- [1] C. Cortes, V. Vapnik, *Support-Vector Networks*. Machine Learning, 20(3):273–297, 1995.
- [2] J. C. Platt, *Sequential Minimal Optimization: A Fast Algorithm for Training Support Vector Machines*. Microsoft Research Technical Report MSR-TR-98-14, 1998.
- [3] B. Schölkopf, A. J. Smola, *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press, 2002.
- [4] D. Dua, C. Graff, *UCI Machine Learning Repository*. University of California, Irvine, 2019. <https://archive.ics.uci.edu>